

Clase 07 (05/05/2026)

1. Transformaciones en 2D

- Definición y notación: Punto, Vector, Marco de referencia
- Transformaciones lineales: analogía con funciones, matrices y producto matriz-vector.
- Matriz de rotación 2D para convertir un punto en el marco de referencia $\{B\}$ rotado respecto de un marco $\{A\}$ un ángulo θ :

$$\begin{bmatrix} x_A \\ y_A \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_B \\ y_B \end{bmatrix}$$

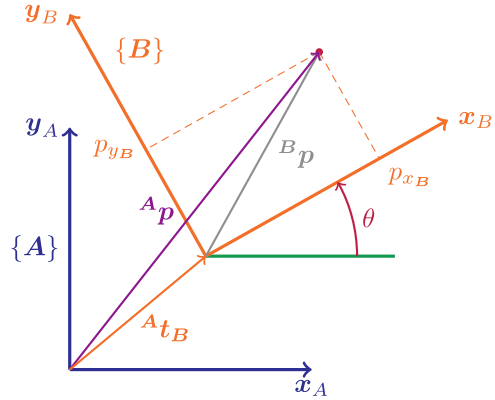
- Traslación 2D: caso no lineal. Coordenadas homogéneas.
- Traslación 2D en coordenadas no homogéneas, de un marco de referencia $\{B\}$ trasladado un vector $\tilde{t}$ respecto de un marco $\{A\}$:

$${}^A \begin{bmatrix} p_x \\ p_y \\ 1 \end{bmatrix} = {}^A \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} {}^B \begin{bmatrix} p_x \\ p_y \\ 1 \end{bmatrix}$$

- Matriz de transformación homogénea:

$${}^A \tilde{p} = \begin{pmatrix} {}^A R_B & {}^A t_B \\ 0 & 1 \end{pmatrix} {}^B \tilde{p}$$

$${}^A \begin{bmatrix} p_x \\ p_y \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & t_x \\ \sin \theta & \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix} {}^B \begin{bmatrix} p_x \\ p_y \\ 1 \end{bmatrix}$$



2. Pose

- Representación, elementos que componen el vector pose en 2 dimensiones: (x, y, θ) .
- Analogía de pose y transformaciones. Cambio de marcos de referencia: ${}^W \tilde{p}_d = {}^W \xi_R {}^R \tilde{p}_d$
- Inverso:

$$\xi^{-1} = \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix}^{-1} = \begin{pmatrix} R^T & -R^T t \\ 0 & 1 \end{pmatrix}$$

- Composición de poses: ${}^A \xi_C = {}^A \xi_B \otimes {}^B \xi_C$

$$\xi_1 \xi_2 = \begin{pmatrix} R_1 R_2 & t_1 + R_1 t_2 \\ 0 & 1 \end{pmatrix}$$

3. Transformaciones en ROS y tf2

3.1. Convenciones: REP103 y REP105

- Convención de unidades: Sistema Internacional
- Regla de la mano derecha: Para marcos de referencia y giros
- Marco ENU y NED
- Nombre de los marcos de referencia con guión bajo. Para plataformas móviles:
 - `base_link`: Fijo a la base del robot.
 - `odom`: Fijo respecto del mundo. Continuo pero con drift.
 - `map`: Fijo respecto del mundo. Con saltos y sin drift (idealmente).
- Representación de rotaciones mediante cuaterniones.

4. tf2: Librería y gestión de transformaciones

- Gestión de transformaciones entre marcos de coordenadas a lo largo del tiempo utilizando una estructura de árbol.
- Cada marco se define con una única transformación respecto de otro marco de referencia (padre). No existen bucles.
- Funciona mediante mensajes de tipo `tf2_msgs/msg/TFMessage` y los topics `\tf` y `\tf_static`. Pero no se utilizan subscriptores o publishers, se trabaja a través de la librería `tf2`.
- Transformaciones estáticas y dinámicas
- Publicación de transformaciones estáticas: `static_transform_publisher`

```
$ ros2 run tf2_ros static_transform_publisher  
  --x <X> --y <Y> --z <Z>  
  --roll <ROLL> --pitch <PITCH> --yaw <YAW>  
  --frame-id <marco_referencia>  
  --child-frame-id <marco_objetivo>
```
- Ver el árbol de transformaciones: `ros2 run rqt_tf_tree rqt_tf_tree`
- Obtener una transformación:

```
$ ros2 run tf2_ros tf2_echo <marco_de_referencia> <marco_objetivo>
```
- Visualizar las transformaciones con Rviz: `ros2 run rviz2 rviz2`

4.1. Publicar transformaciones dinámicas

```
import rclpy
from rclpy.node import Node

# Para publicar la transformación
from tf2_ros import TransformBroadcaster

# Para construir el mensaje de transformación
from geometry_msgs.msg import TransformStamped

class FramePublisher(Node):
    def __init__(self):
        super().__init__('frame_publisher')

        # Inicializar el broadcaster
        self.tf_broadcaster = TransformBroadcaster(self)

        # ..

    def send_tf(self):
        # Cálculo de los elementos de transformación
        # Transformación = Traslación + Rotación
        # ..

        tf = TransformStamped()

        # Marca de tiempo
        tf.header.stamp = self.get_clock().now().to_msg()
        # Marco de referencia (padre)
        tf.header.frame_id = '<marco_referencia>'
        # Marco objetivo (hijo)
        tf.child_frame_id = '<marco_referencia_objetivo>'
        # Marco de referencia (padre)
        tf.header.frame_id = '<marco_referencia>'
        # Marco objetivo (hijo)
        tf.child_frame_id = '<marco_referencia_objetivo>'

        # Traslación
        tf.transform.translation.x = t_x      # Valores calculados
        tf.transform.translation.y = t_y
        tf.transform.translation.z = t_z

        # Rotación
        tf.transform.rotation.x = q_1          # Valores calculados
        tf.transform.rotation.y = q_2
        tf.transform.rotation.z = q_3
        tf.transform.rotation.w = q_0

        # Enviar la transformación
        self.tf_broadcaster.sendTransform(tf)

# ..
```

5. Práctica

5.1. Transformaciones 2D

Dado el siguiente escenario:

- un robot **A** se encuentra en la posición $(2, 1)$ con orientación 30° en coordenadas del *Mundo*
- un robot **B** se encuentra en la posición $(0.5, \frac{\sqrt{3}}{2})$ con orientación 60° en el sistema de coordenadas del robot **A**
- un obstáculo ${}^W O_1 = (0, 3)$ en coordenadas del *Mundo*
- un obstáculo ${}^A O_2 = (\sqrt{3}, -1)$ en coordenadas del robot **A**

Ejercicio 1

Representar todos los robots y sus poses, y todos los sistemas de coordenadas que identifique

Ejercicio 2

¿Cuáles son las coordenadas del obstáculo O_1 en el sistema de coordenadas del robot **A**?

Ejercicio 3

¿Cuáles son las coordenadas del obstáculo O_2 en el sistema de coordenadas del robot **B**?

Ejercicio 4

¿Cuál es la pose del robot **B** en coordenadas del *Mundo*?

Opcional

Pruebe que realizar dos rotaciones consecutivas es lo mismo que realizar una rotación de los ángulos sumados.

5.2. tf2

Ejercicio 5

Crear un archivo launch que recree el escenario de la parte 1 utilizando el nodo `static_transform_publisher` y mediante el comando `tf2_echo` verifique los resultados obtenidos en la parte 1

Ejercicio 6

Implementar un nodo que publique una transformación en movimiento

$$\begin{aligned}x &= R \cdot \cos(t) \\ y &= R \cdot \sin(t)\end{aligned}$$

Para obtener el tiempo t puede utilizar la función `get_clock()` del nodo:

```
seconds, nanoseconds = self.get_clock().now().seconds_nanoseconds()
t = (nanoseconds / 1E9 + seconds)      # convertir a segundos
t = t * (2.0 * np.pi / 4.0)           # 1 vuelta cada 4 segundos
```

Para calcular la orientación, se debe obtener el ángulo de la tangente a la trayectoria

$$\theta = \arctan 2(\cos(t), -\sin(t))$$

y luego formar el cuaternion para la rotación 2D:

$$q = \left(0 \quad 0 \quad \sin \frac{\theta}{2} \quad \cos \frac{\theta}{2}\right)^T$$